# SRAS - Security Robot Automation Systemok

> **System Zarządzania Flotą Robotów Security**

> Dokument opisuje architekturę, roadmapę i plan rozwoju systemu autonomicznych robotów do monitoringu i ochrony obiektów.

---

## 📋 Podsumowanie projektu

**Cel projektu:** Stworzenie zintegrowanego systemu zarządzania flotą robotów security, który umożliwia:

- Autonomiczne patrolowanie obiektów
- Inteligentne wykrywanie zagrożeń
- Sterowanie przez natural language (MCP + LLM)
- Śledzenie historii obiektów i zmian w przestrzeni
- Zarządzanie wieloma robotami (swarm)

---

## 🏗️ Architektura systemu

```
flowchart TB
    subgraph USER["👤 Użytkownik"]
        CHAT["💬 Chat Natural Language"]
        DASHBOARD["📊 Dashboard"]
    end

    subgraph ORCHESTRATION["🧠 Warstwa Orkiestracji"]
        MCP["MCP Server"]
        LLM["LLM (Claude/GPT)"]
        RICHIE["🤖 Richie Mini\n(Orkiestrator)"]
    end
```

```
    subgraph ROS_LAYER["⚙ Warstwa ROS"]
        ROS["ROS 2"]
        ROSBRIDGE["ROS Bridge"]
        NAV2["NAV2\n(Navigation)"]
    end

    subgraph SIMULATION["🎮 Symulacja"]
        ISAC["NVIDIA ISAC\n(Omniverse)"]
        GAZEBO["Gazebo"]
    end

    subgraph ROBOTS["🤖 Roboty"]
        UNITREE["Unitree Go2"]
        SWARM["Swarm\n(małe roboty)"]
        DRONE["Dron"]
        HUMANOID["Humanoid"]
    end

    subgraph SENSORS["📡 Sensory"]
        LIDAR["LIDAR 3D"]
        CAMERA["Kamery"]
        CCTV["CCTV zewnętrzne"]
    end

    subgraph DATA["💾 Dane"]
        VECTORDB["Vector DB"]
        LOGS["Logi zdarzeń"]
        MAPS["Mapy 2D/3D"]
    end

    CHAT --> LLM
    DASHBOARD --> MCP
    LLM --> MCP
    MCP --> ROSBRIDGE
    RICHIE --> MCP
```

```
    ROSBRIDGE --> ROS
    ROS --> NAV2
    ROS --> ISAC
    ROS --> GAZEBO

    ISAC --> UNITREE
    ISAC --> SWARM
    ISAC --> DRONE
    ISAC --> HUMANOID

    UNITREE --> LIDAR
    UNITREE --> CAMERA
    CCTV --> ROS

    LIDAR --> MAPS
    CAMERA --> VECTORDB
    ROS --> LOGS
```

---

## 🔄 Flow sterowania robotem

```
sequenceDiagram
    participant U as 👤 Użytkownik
    participant LLM as 🧠 LLM
    participant MCP as ⚡ MCP Server
    participant ROS as ⚙ ROS Bridge
    participant NAV as 🗺 NAV2
    participant R as 🤖 Robot

    U->>LLM: "Idź do gaśnicy i sprawdź jej stan"
    LLM->>MCP: Interpretacja → wywołanie akcji
    MCP->>ROS: publish_topic(/cmd_nav, {goal: "gasnica_001"})
    ROS->>NAV: Planowanie trasy
    NAV->>R: Komendy ruchu
```

```
    R->>ROS: Status + obraz z kamery
    ROS->>MCP: Dane z sensorów
    MCP->>LLM: Wynik akcji + zdjęcie
    LLM->>U: "Robot dotarł do gaśnicy. Stan: OK, brak zmian."
```

---

## 🎬 Roadmapa DEMO dla Stakeholderów

| # | Demo | Opis | Deliverables |
|---|---|---|---|
| **1** | **Środowisko + Dashboard** | Gotowe środowisko symulacji z dostępem dla Hiro | Film + dostęp do dashboardu (Hiro może sam puścić symulację) |
| **2** | **Unitree Go2** | Przeniesienie na prawdziwego robota | Film z działania robota w rzeczywistości |
| **3** | **Swarm robotów** | Zarządzanie flotą małych robotów | Film ze skoordynowanego działania wielu robotów |
| **4** | **Inteligentne reakcje** | Roboty reagują na zdarzenia | Film z scenariuszami: intruz, torba, eskalacja |
| **5** | **Integracja z KVM** | Połączenie z platformą KVM | Demo end-to-end |
| **6** | **Richie Mini** | Robot orkiestrator całego systemu | Demo asystenta zarządzającego flotą |

---

## ⚠️ Podejście Agile

> **Kolejność faz może się zmieniać!**

> Traktujemy plan zwinnie. W miarę wchodzenia głębiej w tematykę, priorytety będą dostosowywane na podstawie:

> - Feedbacku od stakeholderów

> - Nowych możliwości technologicznych

> - Wyników z poprzednich faz

---

# 🔧 Zasady pracy: Portability

## Filozofia "Install Anywhere"

Pracujemy na zasadzie tworzenia **skryptów automatyzujących**:

- Ułatwiają zarządzanie dependencjami
- Szybka migracja między środowiskami (AWS → TinyBox)
- Instalacja na każdym systemie spełniającym wymagania ISSAC

## Wymagania dla skryptów:

- [ ] Master script = całe środowisko gotowe w 5 min
- [ ] Automatyczna instalacja dependencji
- [ ] Walidacja wymagań systemowych
- [ ] Obsługa: AWS, TinyBox, lokalne maszyny

---

# 📅 Harmonogram Faz

## FAZA 1: Środowisko (Tydzień 1) → DEMO 1

**Cel:** Odbiór i konfiguracja środowiska

- [x] ~~Integracja FoxGlove z ROS~~
- [x] ~~Weryfikacja NAV2 + MCP~~
- [x] ~~Skrypty automatyzacji uruchamiania~~
- [x] ~~Dostęp dla Hiro do dashboardu~~
- [ ] Film demonstracyjny

---

## FAZA 2: Mapowanie 3D (Tydzień 2-3)

**Cel:** Skanowanie i tworzenie map

- [ ] Autonomiczna nawigacja/ mapowanie
- [ ] Skanowanie 3D LIDAR

- [ ] Mapa 2D - widok od góry (blueprint)
- [ ] Integracja zdjęć z mapą → mapa 3D z teksturami
- [ ] Wyszukiwanie obiektów: "Gdzie jest gaśnica?"

```
flowchart LR
    LIDAR["📡 LIDAR"] --> PC["Point Cloud 3D"]
    CAM["📷 Kamera"] --> IMG["Zdjęcia"]
    PC --> MAP3D["🗺️ Mapa 3D"]
    IMG --> MAP3D
    MAP3D --> MAP2D["📋 Widok 2D\n(od góry)"]
    MAP3D --> SEARCH["🔍 Wyszukiwanie\nobiektów"]
```

---

## Faza 2b alternative

- [ ] Autonomiczne podglad na magazyn za pomoca kamer CCTV
  - [ ] issak kamery CCTV + LIDAR jezeli beda ( optional)
  - [ ] Poleczenie z ros bridge → ros → monitoring foxglove/dashboard
  - [ ] VLM przy kamerach CCTV co widzą kamery
- [ ] Triangulacja by okreslic polozenie przedmiotu w magazynie
- [ ] Wyslanie robota do wskazanych przez wynik triangulacji pozycji
- [ ] Robot idzie na pozycje, i za pomoca swojej kamery potwierdza pozycje i przemiot i wysyla alert

## FAZA 3: Dashboard (Tydzień 4-5)

Scope: Widok z kamer CCTV, ekrany = ilosci kamer # - monitoring

Widok zeskanowej przestrzeni lidar

Widok z kamery z kamery robota

Okno chatu/ mcp do ros

**Cel:** Własny dashboard zastępujący FoxGlove

**Komponenty:**

- **Tactical Map** - mapa 2D z pozycjami robotów
- **Overview** - status: aktywne, sieć, bateria
- **Monitoring** - widoki z kamer CCTV + robot
- **Analytics** - statystyki patroli
- **3D Viewer** - interaktywna mapa z teksturami
- **Chat** - sterowanie natural language

---

## FAZA 4: Unitree Go2 (Tydzień 6-7) → DEMO 2

**Cel:** Deployment na prawdziwym robocie

- [ ] Przeniesienie konfiguracji na Unitree Go2
- [ ] Testy w rzeczywistym środowisku
- [ ] Nagranie filmu

Materiały: https://github.com/abizovnuralem/go2_ros2_sdk?tab=readme-ov-file

---

## FAZA 5: Swarm (Tydzień 8-10) → DEMO 3

**Cel:** Zarządzanie flotą robotów

- [ ] 3-4 małe roboty + komunikacja
- [ ] Logika: "ten który jest bliżej"
- [ ] Różne typy: humanoid, dron

```
flowchart TB
    CMD["🗣️ Chodźcie i patrolujcie"] --> BRAIN["🧠 Swarm Inte
lligence"]
    BRAIN --> R1["🤖 Robot 1\n(Hala A)"]
    BRAIN --> R2["🤖 Robot 2\n(Hala B)"]
```

```
    BRAIN --> R3["🤖 Robot 3\n(Wejście)"]
    BRAIN --> D1["🚁 Dron\n(Monitoring)"]
```

---

## FAZA 6: Historia obiektów (Tydzień 11-13) → DEMO 4

**Cel:** Śledzenie zmian + inteligentne reakcje

**Struktura logu zdarzenia:**

```
{
  "timestamp": "2026-01-19T14:30:00Z",
  "object_id": "gasnica_001",
  "object_type": "gaśnica",
  "location": {"x": 12.5, "y": 8.3, "z": 0.9},
  "event_type": "position_change",
  "description": "Gaśnica przesunięta o 0.5m",
  "image_url": "/scans/2026-01-19/gasnica_001.jpg",
  "detected_by": "robot_unitree_01"
}
```

**Przykładowe zapytania:**

| Zapytanie | Odpowiedź |
|---|---|
| "Historia gaśnicy w ostatnim miesiącu?" | "Brak zmian pozycji. Ostatnie skanowanie: dziś 08:00" |
| "Co się zmieniło na półce?" | "17.01 wykryto nowy obiekt: karton" |
| "Zmiany z ostatniego tygodnia" | Lista z timestampami i zdjęciami |
| Przestrzeganie BHP | “Kowalski tydzień temu chodził bez kasku na Hali B”<br>”Gaśnica w Hali C wymaga przeglądu za 2 miesiące” |

---

## FAZA 7: Integracja KVM (Tydzień 14-16) → DEMO 5

**Cel:** Połączenie z platformą KVM

- [ ] Przesyłanie logów i zdarzeń
- [ ] End-to-end workflow
- [ ] Wektoryzacja dla szybkiego wyszukiwania

---

### FAZA 8: Richie Mini (Tydzień 17-19) → DEMO 6

**Cel:** Robot orkiestrator systemu

- [ ] Wgranie systemu na Richie Mini
- [ ] Centralny hub zarządzający
- [ ] Komunikacja głosowa

**Przykładowe komendy:**

- "Richie, status wszystkich robotów"
- "Richie, wyślij robota do hali B"
- "Richie, historia gaśnicy?"

---

## 🎯 Stretch Goals

### Ręka robotyczna + Haptyka

- [ ] Symulacja ręki w ISAC
- [ ] Uczenie: otwieranie klamki, chwytanie
- [ ] Sterowanie z joysticka haptycznego
- [ ] Stream interwencyjny: zdalne sterowanie

https://x.com/IlirAliu_/status/2013173665412547026?s=20

### Forensic Studio

- [ ] przegladanie materialow dowodowych

### Osint Intergracja

- [ ] Informacje o odwiedzajacych, skanowanie

---

## 🔮 Dalsze kierunki rozwoju

> Te kierunki będą eksplorowane w miarę postępu projektu.

### Vision Language Models (VLM)

- Głębsza integracja z modelami wizyjno-językowymi
- Kontekstowe rozumienie scen
- Automatyczne raporty z patroli

### Zaawansowane scenariusze

**Przykład: Zostawiona torba**

```
flowchart TB
    A["🤖 Robot wykrywa nowy obiekt"] --> B["❓ Kto był w pob
liżu?"]
    B --> C["👤 Użytkownik A zostawił torbę"]
    C --> D{"Ocena ryzyka"}
    D -->|Niskie| E["📝 Log + monitoring"]
    D -->|Wysokie| F["🚨 Alert + eskalacja"]
```

**Inne scenariusze:**

- Nietypowe zachowanie (ktoś kręci się bez celu)
- Dostęp do stref zastrzeżonych
- Wykrycie sytuacji awaryjnych

---

## 👥 Zespół

| Osoba | Odpowiedzialność |
|---|---|
| **Jacek Caban** | FoxGlove, NAV2+MCP, skrypty portability |

| Osoba | Odpowiedzialność |
|---|---|
| **Piotr Gerke** | Dashboard, kamery, sterowanie NL, demo, Richie Mini |

---

## 🛠️ Stack technologiczny

| Kategoria | Technologie |
|---|---|
| **Robot OS** | ROS 2, ROS Bridge |
| **Symulacja** | NVIDIA ISAC (Omniverse), Gazebo |
| **Nawigacja** | NAV2 (Navigation Framework) |
| **AI/LLM** | MCP, Claude/GPT, VLM |
| **Dashboard** | FoxGlove → własny |
| **Infrastruktura** | AWS (G6 2xlarge), TinyBox, DCV |
| **Roboty** | Unitree Go2, Richie Mini, Swarm |
| **Dane** | Vector DB, KVM |

---

## 📎 Referencje

- **ROS MCP Server:** [GitHub repo]
- **NAV2:** https://docs.nav2.org/
- **FoxGlove:** https://foxglove.dev/
- **Źródło:** Fireflies transcript (19.01.2026)